When to Write a Kubernetes Operator

and How to Do It Right

The trap

“Writing a Kubernetes Operator sounds great…”

…but often:

- YAML would have been enough
- complexity sneaks in
- maintenance cost explodes

So when *does* an Operator make sense?

When you need:

- relationships between resources
- invariants (“if X exists, Y must exist”)
- lifecycle & cleanup
- continuous enforcement

When you *don't* need one

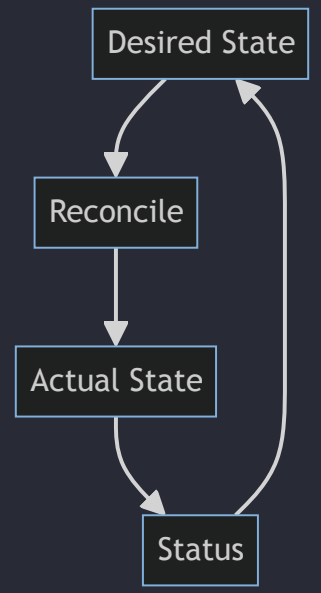
If it's:

- static resources
- one-off generation
- simple templating

➡️ **Just use YAML.**

What an Operator really is

No magic. Just a loop.



What makes a good Operator

- simple reconciliation logic
- idempotent behavior
- clear, meaningful status
- boring, predictable outcomes

What the Operator does not do

- no workflows
- no step-by-step logic
- no hidden state

It only:

- reads the world
- enforces a contract

The line to draw

- one resource → YAML
- relationships → Operator
- time / lifecycle / invariants → Operator

Let's scaffold an operator...

```
operator-sdk init --domain "$DOMAIN" --repo "$REPO"
```

... And Create a Resources and a Controller

```
operator-sdk create api \
  --group kitchen \
  --version v1alpha1 \
  --kind Teapot \
  --resource \
  --controller
```

Reconcile is the Operator

```
func (r *TeapotReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    // read current state
    // check dependencies
    // update status
    return ctrl.Result{}, nil
}
```

Demo

Takeaways

- Operators are powerful - and expensive
- Write them - and code in general - rarely
- Keep them small
- Make status your UX